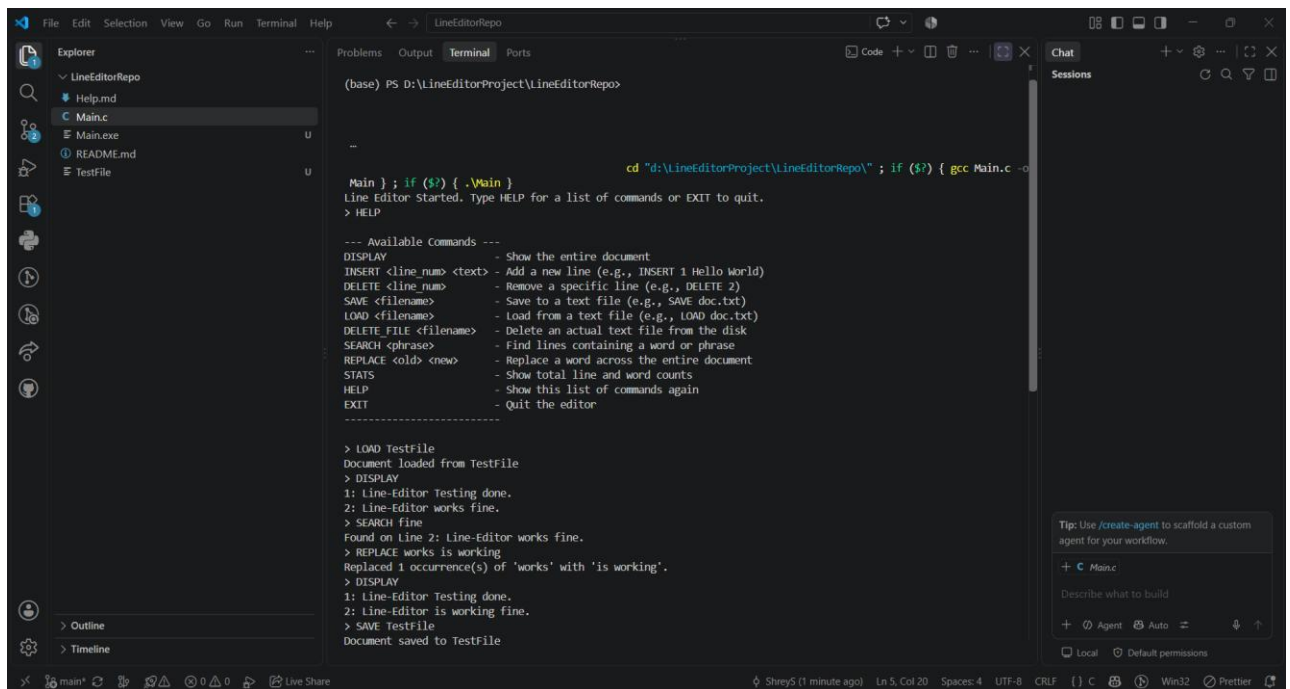


Output:

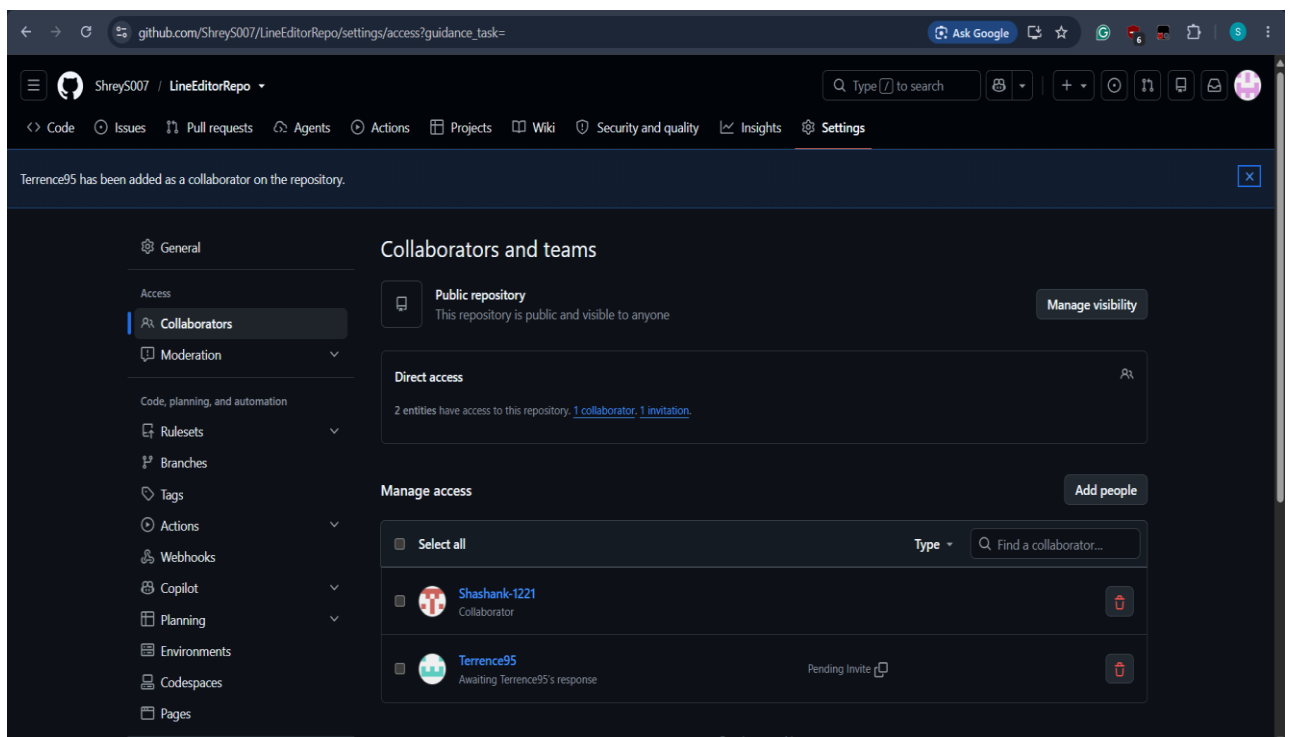


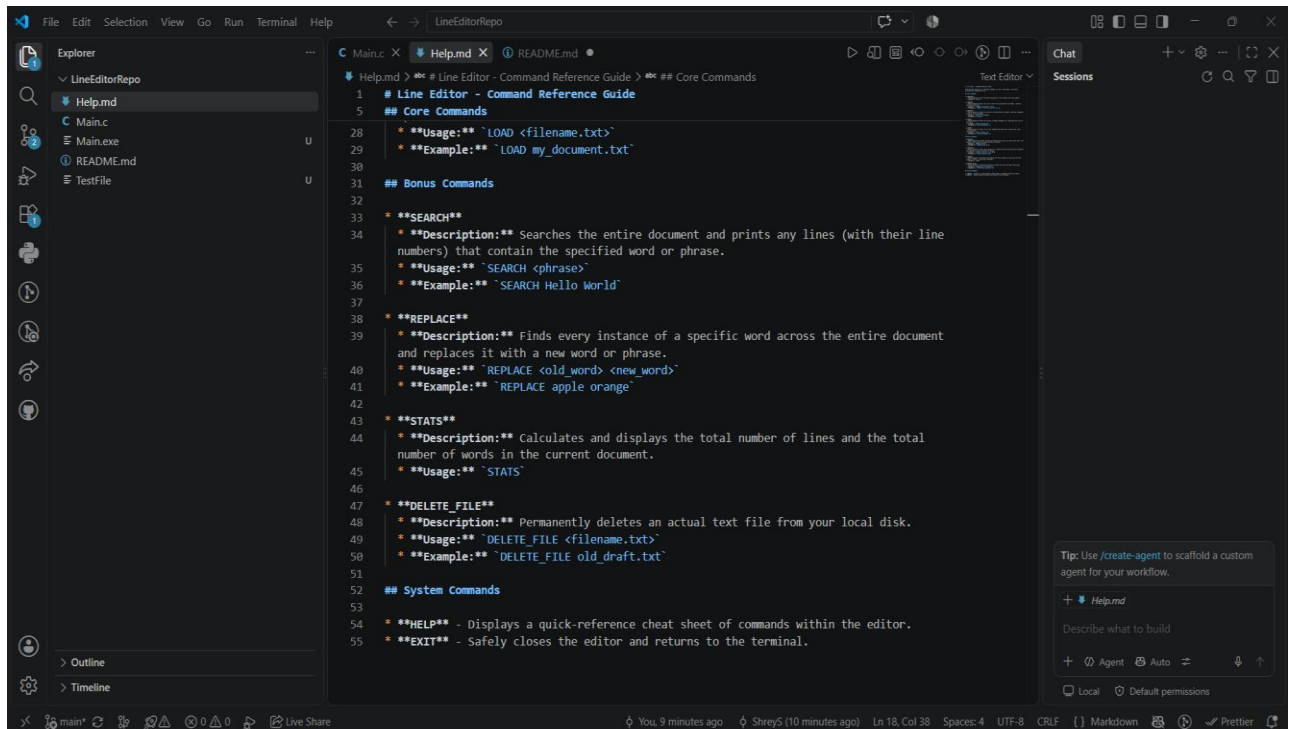
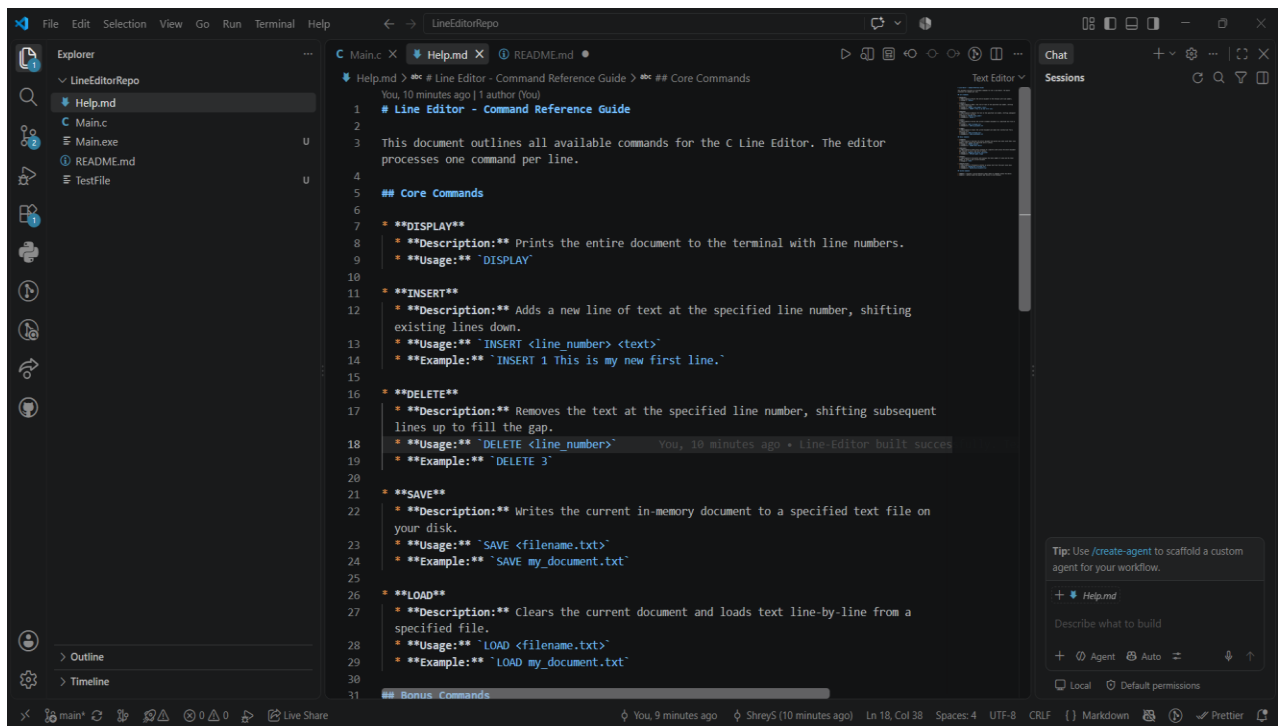
```
(base) PS D:\LineEditorProject\LineEditorRepo> cd "d:\LineEditorProject\LineEditorRepo" ; if ($?) { gcc Main.c -o Main }

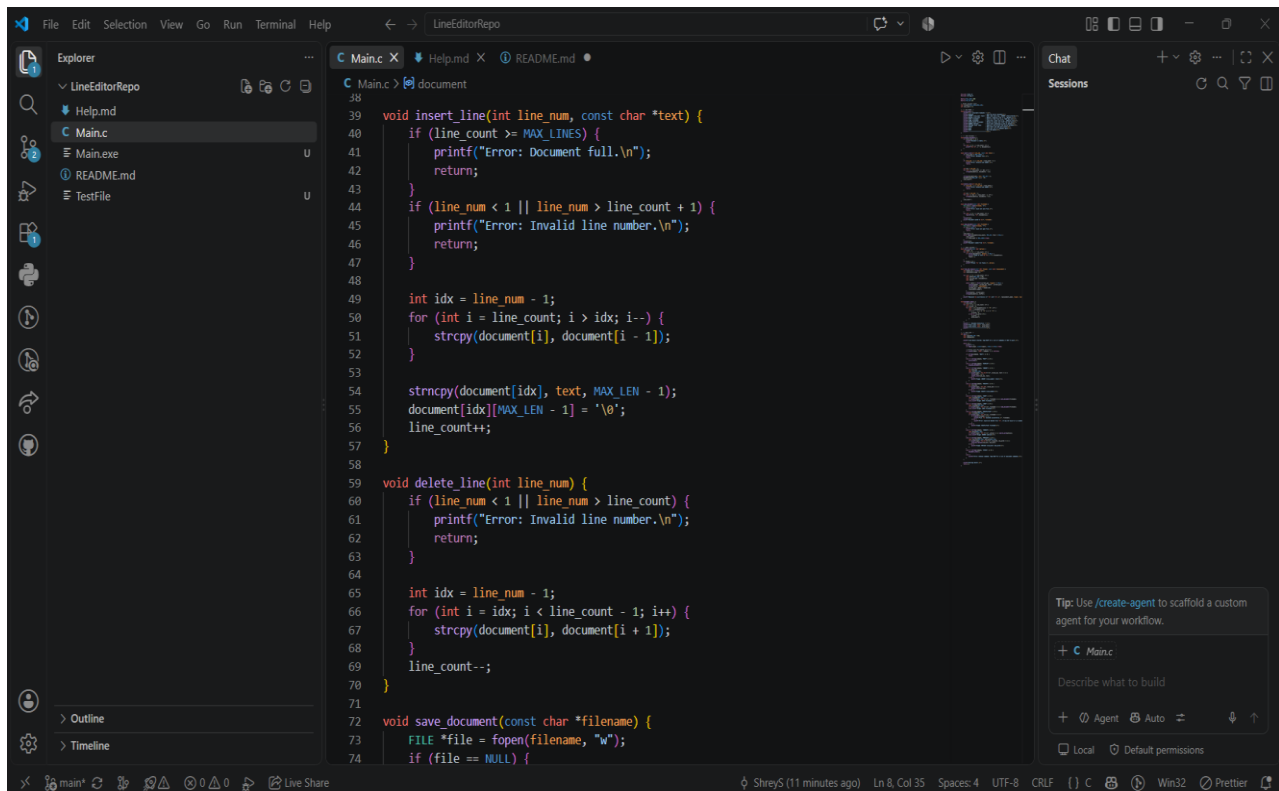
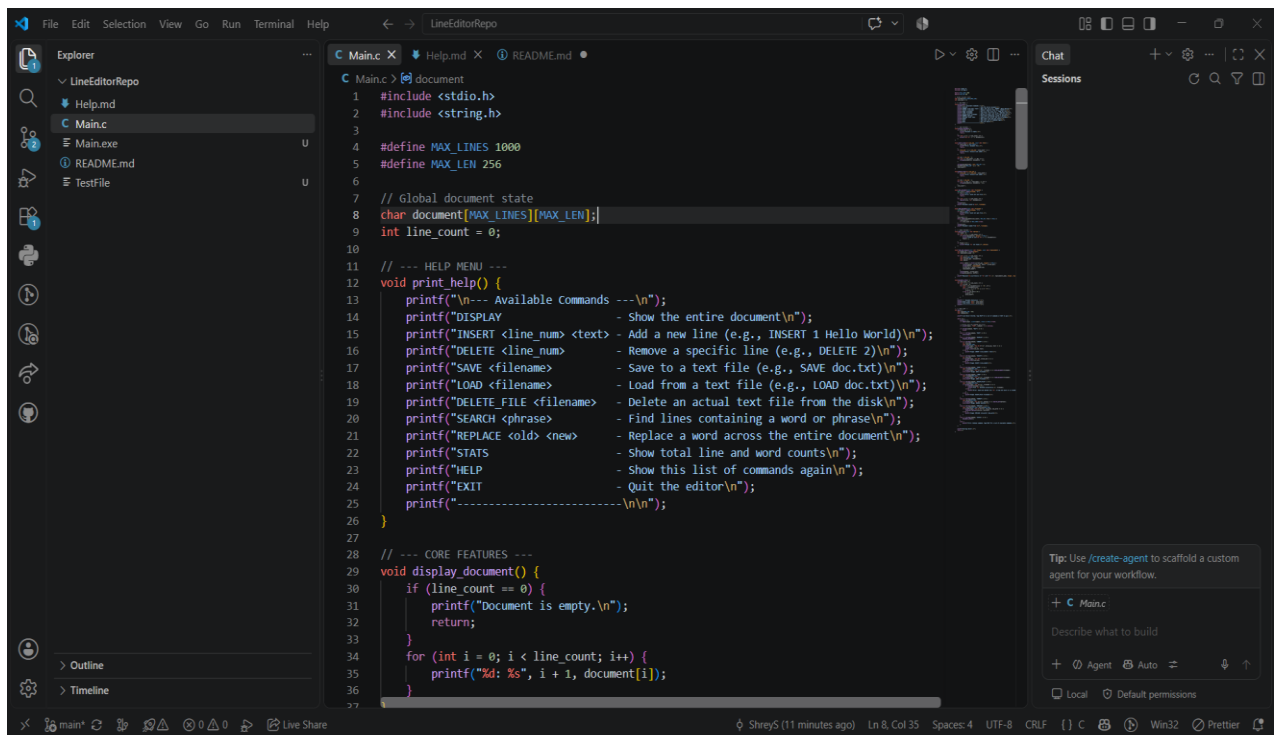
Main ) : if ($?) { .\Main }
Line Editor Started. Type HELP for a list of commands or EXIT to quit.
> HELP

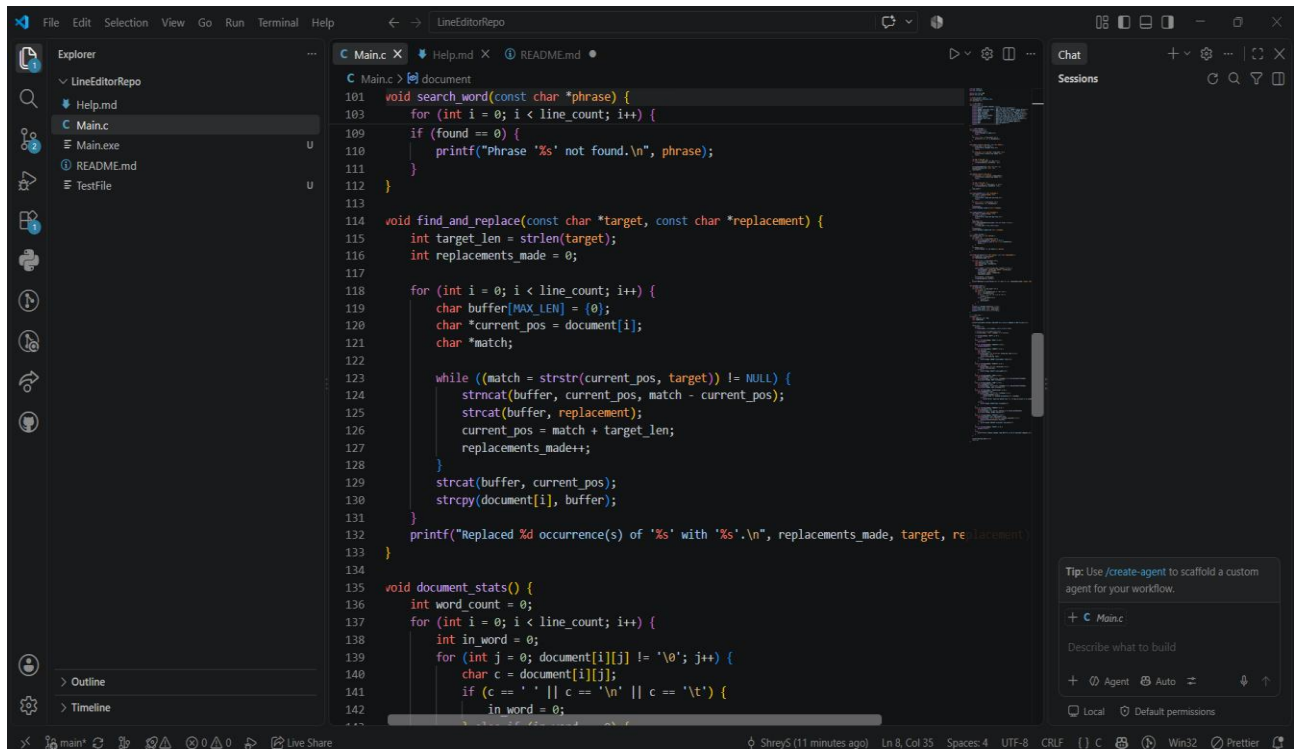
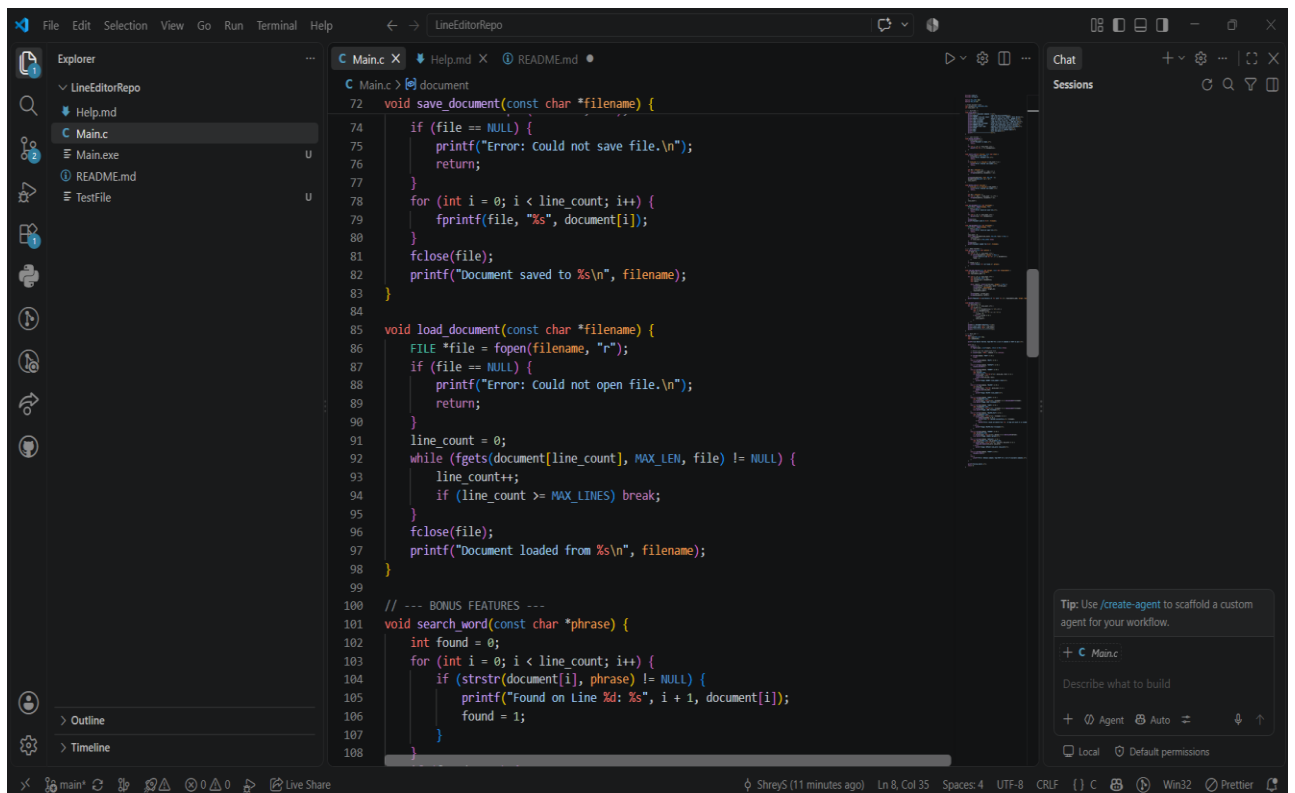
--- Available Commands ---
DISPLAY          - Show the entire document
INSERT <line_num> <text> - Add a new line (e.g., INSERT 1 Hello World)
DELETE <line_num> - Remove a specific line (e.g., DELETE 2)
SAVE <filename>   - Save to a text file (e.g., SAVE doc.txt)
LOAD <filename>   - Load from a text file (e.g., LOAD doc.txt)
DELETE_FILE <filename> - Delete an actual text file from the disk
SEARCH <phrase>   - Find lines containing a word or phrase
REPLACE <old> <new> - Replace a word across the entire document
STATS            - Show total line and word counts
HELP             - Show this list of commands again
EXIT             - Quit the editor
-----

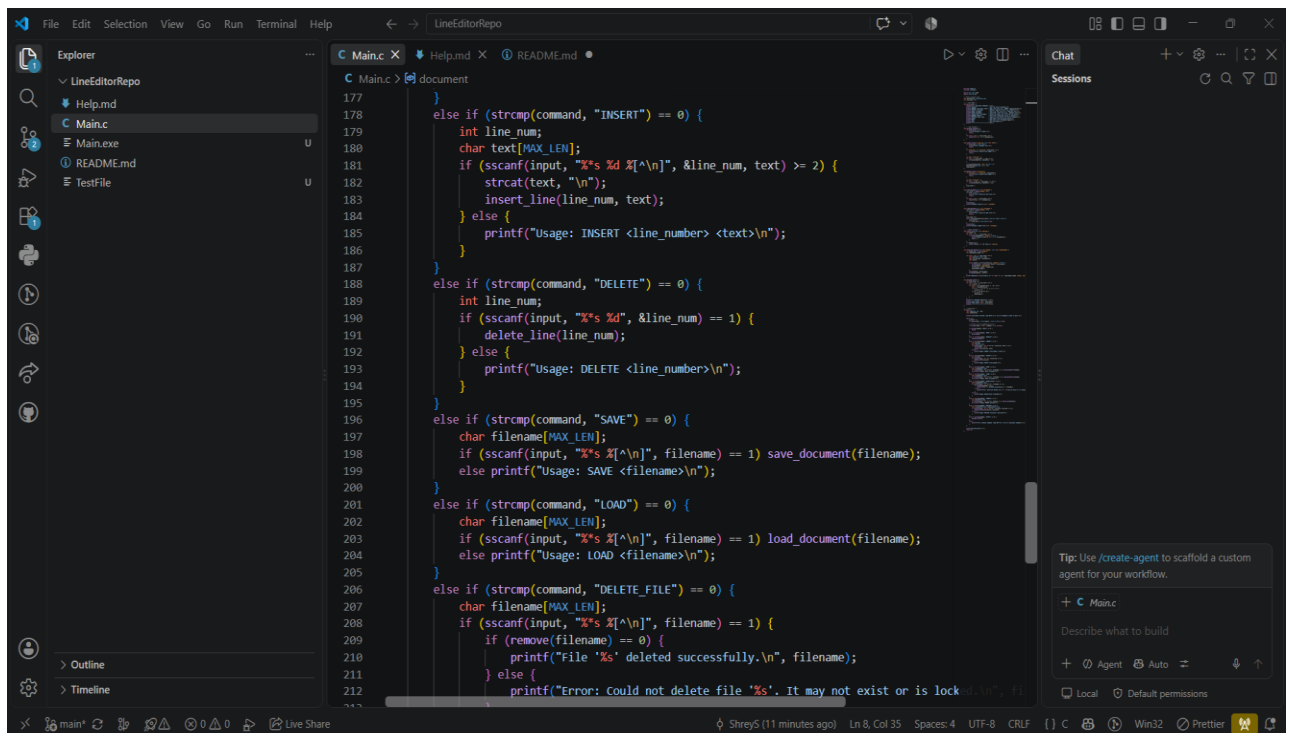
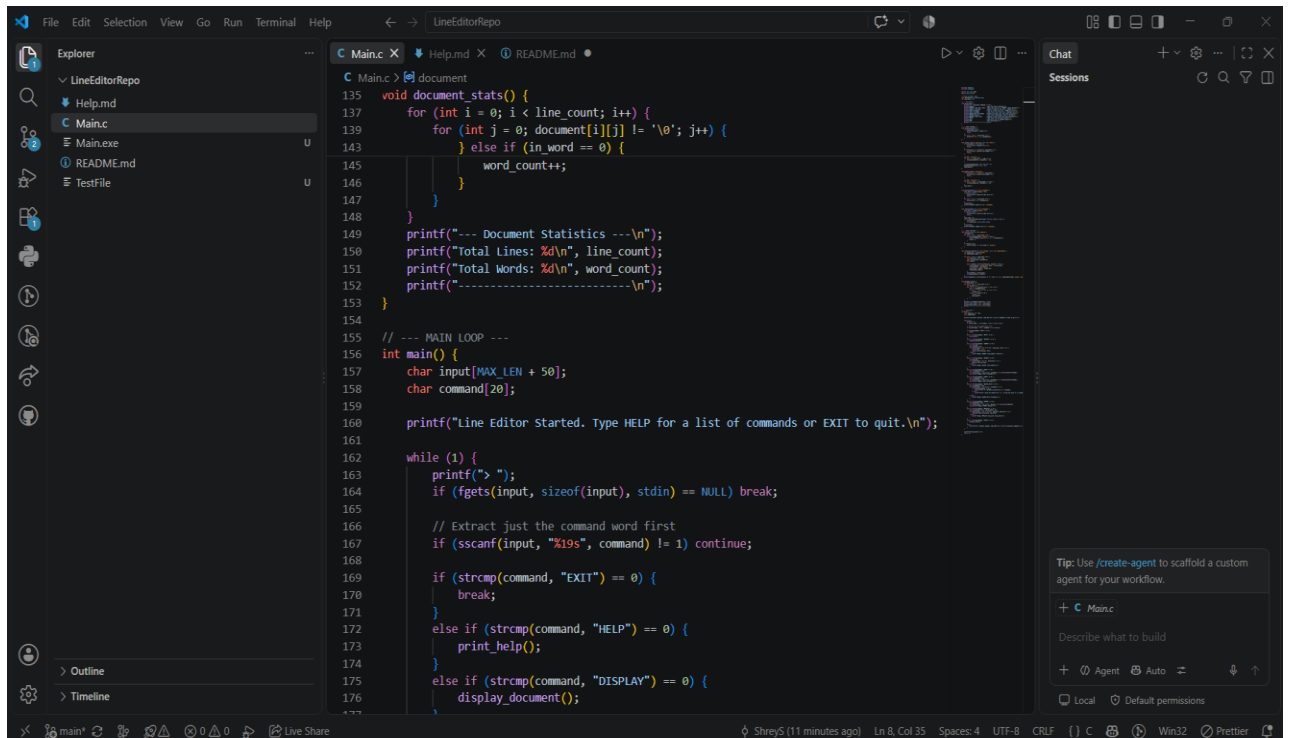
> LOAD TestFile
Document loaded from TestFile
> DISPLAY
1: Line-Editor Testing done.
2: Line-Editor works fine.
> SEARCH fine
Found on line 2: Line-Editor works fine.
> REPLACE works is working
Replaced 1 occurrence(s) of 'works' with 'is working'.
> DISPLAY
1: Line-Editor Testing done.
2: Line-Editor is working fine.
> SAVE TestFile
Document saved to TestFile
```

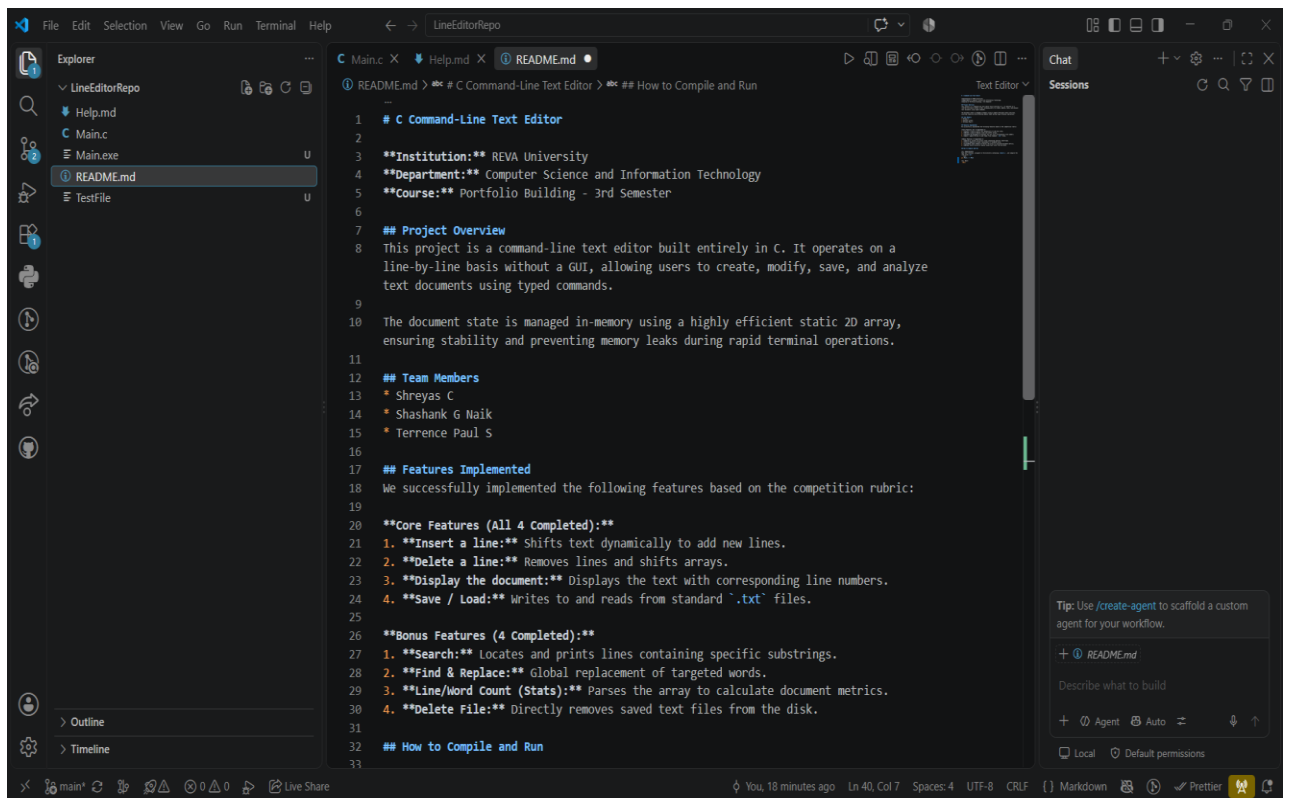
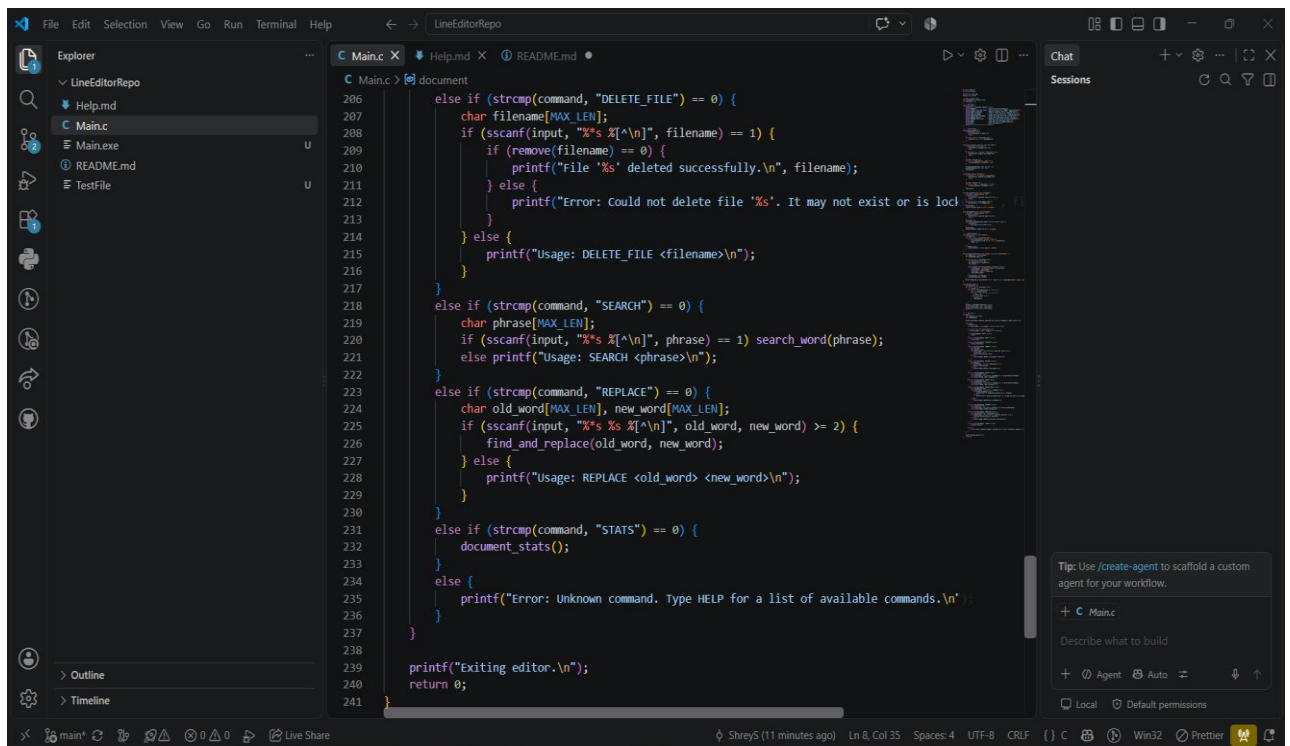


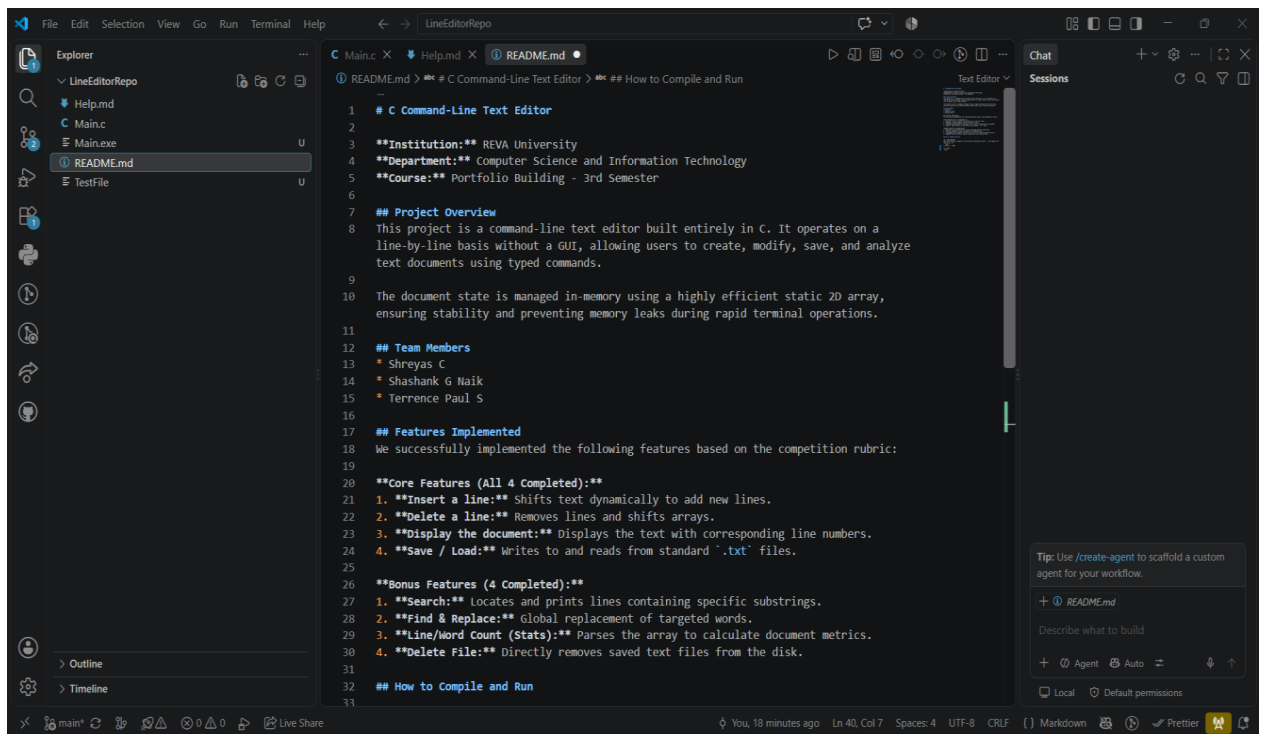
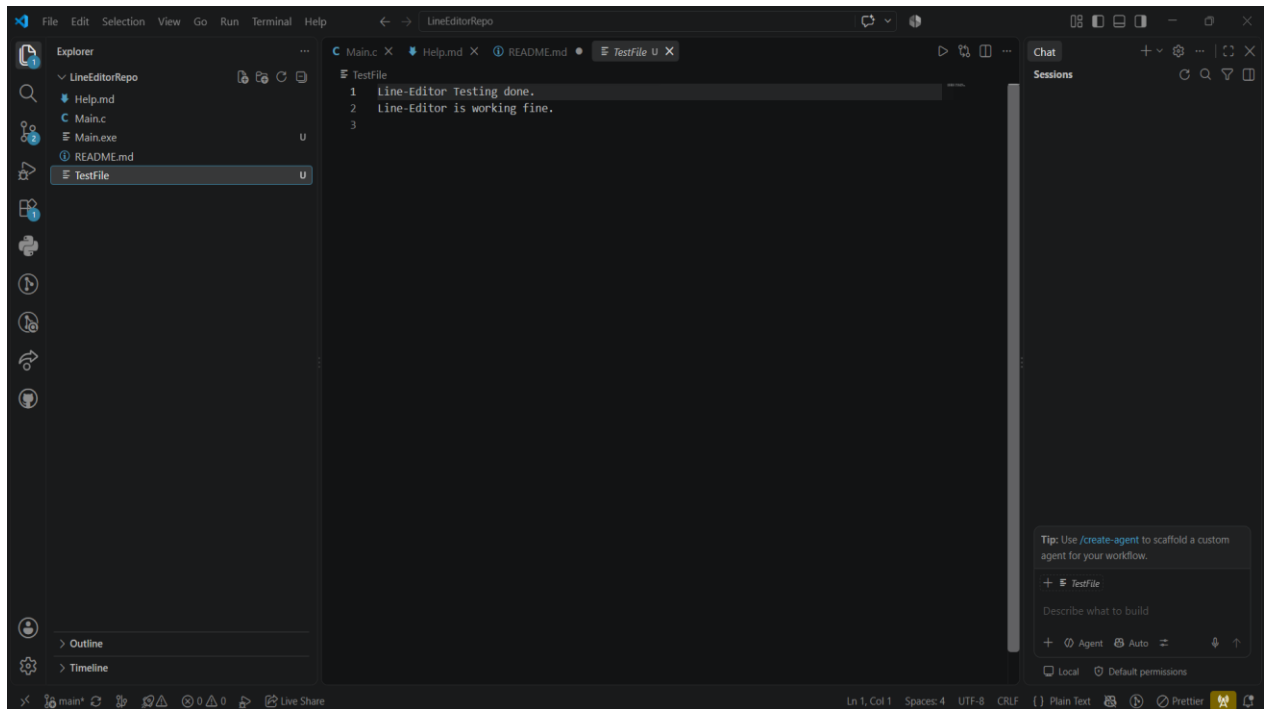












```
#include <stdio.h>
#include <string.h>
#define MAX_LINES 1000
#define MAX_LEN 256
char document [MAX_LINES][MAX_LEN];
int line_count = 0;

void print_help () {
printf("\n-- Available commands ==-\n");
printf("Display - Show the entire document\n");
printf("INSERT <line-num> <text> - Add a new line\n");
printf("Eg., INSERT 1 Hello World\n");
printf("DELETE <line-num> - remove a specific line\n");
printf("Eg., DELETE 2\n");
printf("SAVE <filename> - Save to a text file\n");
printf("Eg., SAVE doc.txt\n");
printf("LOAD <filename> Load from a text file\n");
printf("Eg., LOAD doc.txt\n");
printf("DELETE-FILE <filename> - Delete an actual text file from the disk\n");
printf("SEARCH <phrase> - find lines containing a word or phrase\n");
printf("REPLACE <old> <new> - replace a word across the entire document\n");
printf("STATS - Show total line & word counts\n");
printf("HELP - Show this list of commands again\n");
printf("\n-- -- --\n");
}

void display_document () {
if (line_count == 0) {
printf("Document is empty\n");
return;
}
```

Date:

```

for (int i=0; i < line_count; i++) {
    printf ("%d: %s", i+1, document[i]);
}

```

```

}
void insert_line (int line_num, const char* text)
{

```

```

    if (line_count >= MAX_LINES) {
        printf ("Error: Document full.\n");
        return;
    }

```

```

    if (line_num < 1 || line_num > line_count + 1) {
        printf ("Error: Invalid line number.\n");
        return;
    }

```

```

    int idx = line_num - 1;
    for (int i = line_count; i > idx; i--) {
        strcpy (document[i], document[i-1]);
    }

```

```

    strcpy (document[idx], text, MAX_LEN-1);
    document[idx][MAX_LEN-1] = '\0';
    line_count++;
}

```

```

void delete_line (int line_num) {

```

```

    if (line_num < 1 || line_num > line_count) {
        printf ("Error: Invalid line number.\n");
        return;
    }

```

```

    int idx = line_num - 1;
    for (int i = idx; i < line_count - 1; i++) {
        strcpy (document[i], document[i+1]);
    }
    line_count--;
}

```

```

void save_document (const char * filename) {
    FILE * file = fopen (filename, "w");
    if (file == NULL) {
        printf ("Error: Could not save file.\n");
        return;
    }
    for (int i=0; i<line_count; i++) {
        fprintf (file, "%s", document[i]);
    }
    fclose (file);
    printf ("Document saved to %s\n",
           filename);
}

```

```

void load_document (const char * filename) {
    FILE * file = fopen (filename, "r");
    if (file == NULL) {
        printf ("Error");
        return;
    }

```

```

    line_count = 0;
    while (fgetc)
        line_count++;
    if (line_count >= MAX_LINES) break;
}

```

```

    fclose (file);
    printf ("...");
}

```

```

void search_word (const ) {
    int found = 0;
    for ( )
        if (strstr (doc[i], phrase) != NULL) {

```

```

    printf(" ");
    found = 1;
    if (found == 0) {
        printf(" ");
    }
}

```

```

void find & replace (const, const) {
    int target = strlen (tar);
    int replacements = 0;
    for ( )

```

```

    write
    printf ( )
}

```

// BONUS

```

void document_stats ( ) {
    int word_count = 0;
    for ( ) {
        int in_word = 0;
        for ( )

```

```

}
int main ( ) {
    switch (ch) {
        case 1 :

```

case 9 :-

return ;

}